Database Development and Design (SOE201)

Lecture 5b: Introduction to Query Optimisation 2

Dr. Wei Wang

AIAT

Learning Outcomes

- Introduction to Query Optimisation
  - Cost Estimation
  - Cost-based Optimisation
  - Join Order Optimisation
  - Heuristic Optimisation

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Catalog Information for Cost Estimation

- n_r: number of tuples in relation r.
- b_r: number of blocks containing tuples of r.
- l_r: size of tuple of r.
- f_r: blocking factor of r; i.e. number of tuples of r that fit into one block.
- $V(A, r)$: number of distinct values that appear in r for attribute A; same as size of $\prod_A(r)$.
- If tuples of r are stored contiguously in file, then:

$$b_r = \left\lceil \frac{n_r}{f_r} \right\rceil$$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Estimation of Size of Selection

- $\sigma_{A=v}(r)$
  - n_r / $V(A,r)$: number of records that satisfy selection
  - Equality condition on key attribute: *size estimate* = 1 if it exists
- $\sigma_{A \leq v}(r)$ (case of $\sigma_{A \geq v}(r)$ is symmetric)
  - Let c denote estimated number of tuples satisfying condition.
  - Let min(A,r) and max(A,r) denote lowest and highest values for attribute A.
  - If min(A,r) and max(A,r) are available in catalog
    - c = 0 if v < min(A,r)
    - $c = n_r \cdot \frac{v - \min(A,r)}{\max(A,r) - \min(A,r)}$
  - If histograms available, can refine above estimate
  - In absence of statistical information c is assumed to be $n_r / 2$.

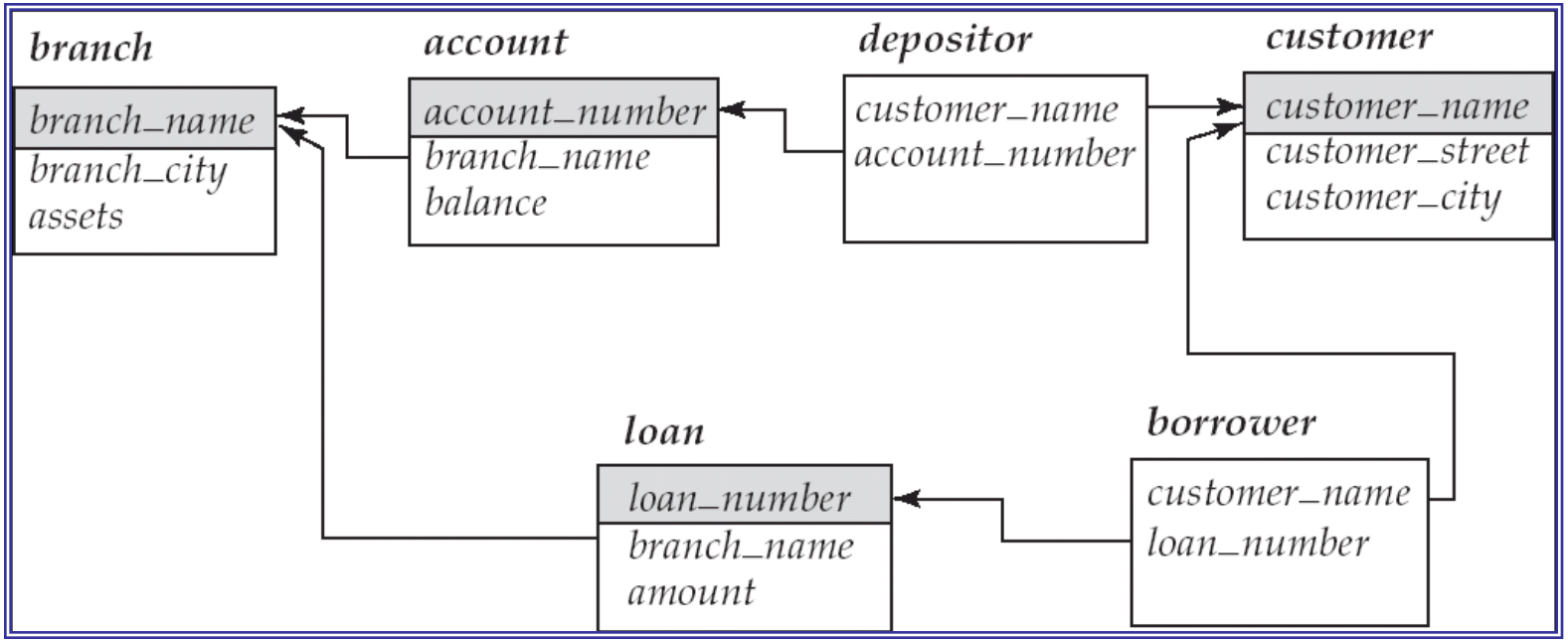
Xi'an Jiaotong-Liverpool University 西交利物浦大学

Estimation of Size of Joins

- Cartesian product $r \times s$ contains $n_r . n_s$ tuples; each tuple occupies $s_r + s_s$ bytes.
- If $R \cap S = \emptyset$ (same as Cartesian product), then $r \bowtie s$ is same as $r \times s$.
- If $R \cap S$ is key for R, then tuple of s will join with at most one tuple from r
  - Therefore, number of tuples in $r \bowtie s$ is no greater than number of tuples in s.
- If $R \cap S$ is foreign key in S referencing R, then number of tuples in $r \bowtie s$ is exactly same as number of tuples in s.
  - Case for $R \cap S$ being foreign key referencing S is symmetric.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Banking Example



- Number of records of *customer*: 10,000
- Number of blocks of *customer*: 400
- Number of records of *depositor*: 5,000
- Number of blocks of *depositor*: 100
- Example query: "*depositor* ⋈ *customer*", *customer_name* in *depositor* is foreign key (referencing *customer*), hence, result has exactly $n_{depositor}$ tuples, which is 5,000.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Estimation of Size of Joins cont'd

- If $R \cap S = \{A\}$ is not key for R or S.
  If assume that every tuple t in R produces tuples in $R \bowtie S$, number of tuples in $R \bowtie S$ is estimated to be:

$$\frac{n_r * n_s}{V(A,s)}$$

  If reverse is true, estimate obtained will be:

$$\frac{n_r * n_s}{V(A,r)}$$

Lower of these two estimates is *probably more accurate*.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Join Operation: Running Example

- Running example
  - compute size estimates for "*depositor* ⋈ *customer*" without using information about foreign keys.
- Catalog information:
  - $n_{customer}$ = 10,000; $f_{customer}$ = 25, which implies that $b_{customer}$ =10,000/25 = 400.
  - $n_{depositor}$ = 5000; $f_{depositor}$ = 50, which implies that $b_{depositor}$ = 5,000/50 = 100.
  - *V(customer_name, depositor)* = 2,500
    - Implies that, on average, each customer has two accounts.
  - *V(customer_name, customer) = 10,000 (primary key)*
  - Two estimates are:
    - $\frac{n_r * n_s}{V(A,s)}$ = 5,000 * 10,000/2,500 = 20,000
    - $\frac{n_r * n_s}{V(A,r)}$ = 5,000 * 10,000/10,000 = 5,000
  - Choose lower estimate, which in this case, is same as earlier estimate using foreign keys (*customer_name* in *depositor* is foreign key on *customer, so size is 5,000*).

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Size Estimation for Other Operations

- Projection: estimated size of $\Pi_A(r) = V(A,r)$
- Set operations
  - For unions/intersections of selections on same relation: rewrite and use size estimate for selections. Let s_i denote size of $\sigma_{\theta i}(r)$.
    - Conjunction: $n_r * \frac{s_1 * s_2 * \cdots * s_n}{n_r^n}$
    - Disjunction: $n_r * (1 - (1 - \frac{s_1}{n_r}) * (1 - \frac{s_2}{n_r}) * \cdots * (1 - \frac{s_n}{n_r}))$
  - For operations on different relations:
    - estimated size of $r \cup s$ = size of r + size of s.
    - estimated size of $r \cap s$ = minimum size of r and size of s.
    - estimated size of $r - s$ = r.
    - *All three estimates may be quite inaccurate, but provide upper bounds on sizes.*

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Estimation of Number of Distinct Values in Selection

- If θ forces A to take specified value: $V(A,\sigma_\theta(r)) = 1$, e.g. $A = 3$
- If θ forces A to take on one of specified set of values: $V(A,\sigma_\theta(r))$ = number of specified values, e.g. $(A = 1 \vee A = 3 \vee A = 4)$
- If selection condition θ is of form: $A\ op\ v$ (op is >, <, etc); s is selectivity of selection.

  $V(A,\sigma_\theta(r)) = V(A,r) * s$
- In all other cases: use approximate estimate of $\min(V(A,r), n_{\sigma\theta(r)})$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Estimation of Number of Distinct Values in Join

- If all attributes in A are from r, then estimated
  $V(A, r \bowtie s) = \min(V(A,r), n_{r \bowtie s})$
- If A contains attributes $A1$ from r and $A2$ from s, then estimated
  $V(A, r \bowtie s) = \min(V(A1,r) * V(A2 - A1,s), V(A1 - A2,r) * V(A2,s), n_{r \bowtie s})$

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Cost-Based Join Order Optimisation

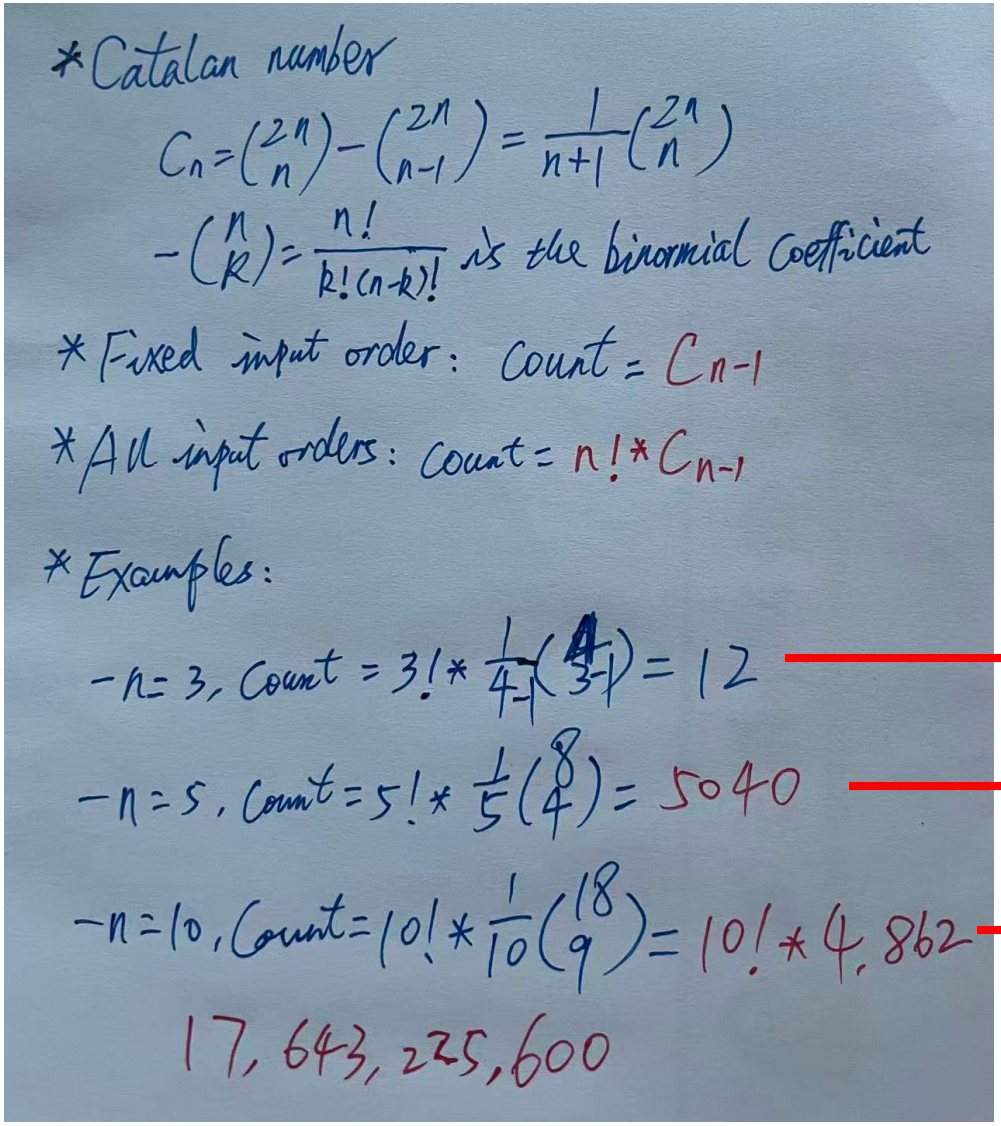
- Consider finding best join-order for

  $r_1 \bowtie r_2 \bowtie \ldots r_n$

- There are $(2(n-1))!/(n-1)!$ different join orders for above expression.
  - With $n = 3$, number is 12
  - With $n = 5$, number is 1,680
  - With $n = 10$, number is greater than 17 billion!
- No need to generate all join orders.
- Using dynamic programming, least-cost join order for any subset of $\{r_1, r_2, \ldots r_n\}$ is computed only once and stored for future use.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Proof of Count of Join Orders



1. https://pi.math.cornell.edu/~karola/dimex/catalan.pdf
2. https://brilliant.org/wiki/catalan-numbers/

Proof from Gemini 3

give a brief Proof of Count of Join Orders, provide the number when n is 3, 5, and 10.

In relational database optimization, join trees are typically modeled as full binary trees where leaf nodes are relations and internal nodes are binary join operations. Assuming standard inner joins (which are **commutative** and **associative**), the total number of possible join orders considers both the permutation of relations and the grouping (parenthesization) of joins. Querify Labs

Brief Proof of Join Orders (Bushy Trees)

To find the total number of valid join orders for n relations $(R_1, R_2, \ldots, R_n)$, we break the problem into two independent choices:

1. **Permutation of Leaf Nodes ($n!$):**
   Because join operations are commutative, the n input relations can be arranged in any sequence at the leaves of the tree. The number of ways to arrange n unique items is given by the factorial $n!$.
2. **Tree Shapes / Parenthesization (C_{n-1}):**
   Because join operations are associative, a fixed sequence of n leaves can be grouped using different binary tree topologies (bushy trees). The number of full binary trees with n leaves is counted by the $(n-1)$-th **Catalan number**, denoted as: Querify Labs

$$C_{n-1} = \frac{1}{n}\binom{2n-2}{n-1}$$

Multiplying these two independent components yields the total formula for general **bushy join orders**: Querify Labs

$$\text{Total Join Orders} = n! \times C_{n-1}$$

*(Note: If an optimizer restricts itself to **Left-Deep Trees**—where the right child of every join is always a base relation—the tree shape is fixed, reducing the search space to just $n!$).*

Counts for $n = 3, 5,$ and 10

n (Relations)	Left-Deep Trees (n!)	General Bushy Trees (n!×Cn−1)
3	6	12
5	120	1,680
10	3,628,800	17,643,225,600

Dynamic Programming in Join Order Optimisation

- To find *best plan* (join tree) for set of *n* relations:
  - Consider all possible plans of form: $S_1 \bowtie (S - S_1)$, where S_1 is any non-empty subset of S.
  - Recursively compute cost for joining subsets of S to find cost of each plan. Choose cheapest of alternatives.
  - *Base case* for recursion: single relation access plan
    - e.g. find best selection strategy for particular relation R_i
  - When plan for any subset is computed, store and reuse it when required again, instead of re-computing it.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Join Order Optimisation Algorithm

```
// initialise bestplan[S].cost to ∞
procedure findbestplan(S)
    if (bestplan[S].cost ≠ ∞)
            return bestplan[S]
    // else bestplan[S] has not been computed earlier, compute it now
    if (S contains only one relation)
            set bestplan[S].plan and bestplan[S].cost based on best way
            of accessing S  /* Using selections on S, e.g. indices on S */
    else for each non-empty subset S1 of S such that S1 ≠ S
            P1= findbestplan(S1)
            P2= findbestplan(S - S1)
            A = best algorithm for joining results of P1 and P2
            cost = P1.cost + P2.cost + cost of A
            if cost < bestplan[S].cost
                    bestplan[S].cost = cost
                    bestplan[S].plan = "execute P1.plan; execute P2.plan;
                                        join results of P1 and P2 using A"
    return bestplan[S]
```

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Cost of Join Order Optimisation

- With dynamic programming, time complexity of optimisation with bushy trees is $O(3^n)$.
  - With $n = 10$, this number is 59,000 instead of 176 billion!
- Space complexity is $O(2^n)$ as number of subsets of S is 2^n
- Although both numbers still increase rapidly with n, commonly occurring joins usually have less than 10 relations, and can be handled easily.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Choice of Evaluation Plans

- Must consider *interaction* of evaluation techniques when choosing evaluation plans
  - *Choosing cheapest algorithm for each operator independently may not yield best overall algorithm*
    - Merge-join may be costlier than hash-join, but may provide sorted output which reduces cost for outer level aggregation.
    - Nested-loop family joins may provide opportunity for pipelining
- Practical query optimisers incorporate elements of following two broad approaches:
  - Search all plans and choose best in cost-based fashion.
  - Use heuristics.

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Cost-Based Optimisation with Equivalence Rules

- General-purpose cost-based optimiser based on equivalence rules
  - Easy to extend optimiser with new rules to handle different query constructs
    - But procedure to enumerate all equivalent expressions is very expensive
  - Many optimisers follow approach based on
    - Heuristic transformations to handle constructs other than joins
    - Applying cost-based join order selection algorithm to subexpressions involving only joins and selections
- To make the approach work efficiently requires following:
  - Space-efficient representation of expressions
  - Efficient techniques for detecting duplicate derivations of same expression
  - Dynamic programming based on memoisation
  - Avoid generating all possible equivalent plans

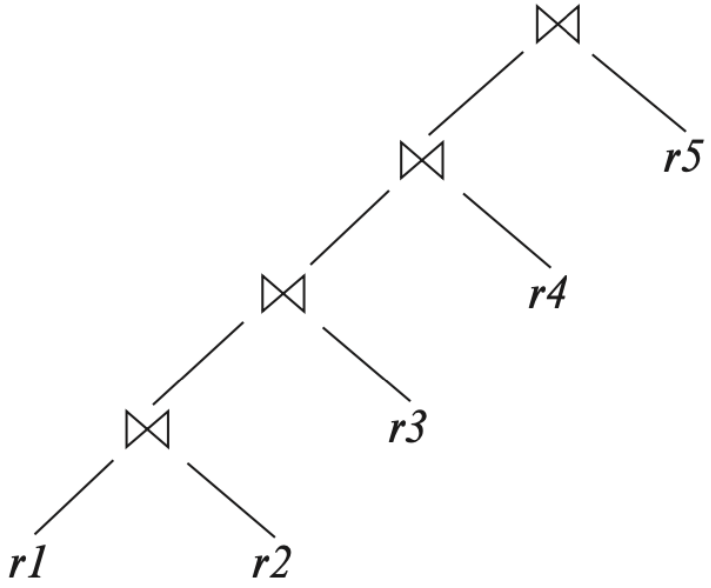
Heuristic Optimisation

- Cost-based optimisation is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce number of choices that must be made in cost-based fashion.
- Heuristic optimisation transforms query-tree by using set of rules that typically (but not in all cases) improve execution performance:
  - Perform selection early (reduces number of tuples)
  - Perform projection early (reduces number of attributes)
  - Perform most restrictive selection and join operations (i.e. with smallest result size) before other similar operations.
- Some systems use only heuristics, others combine heuristics with partial cost-based optimisation.

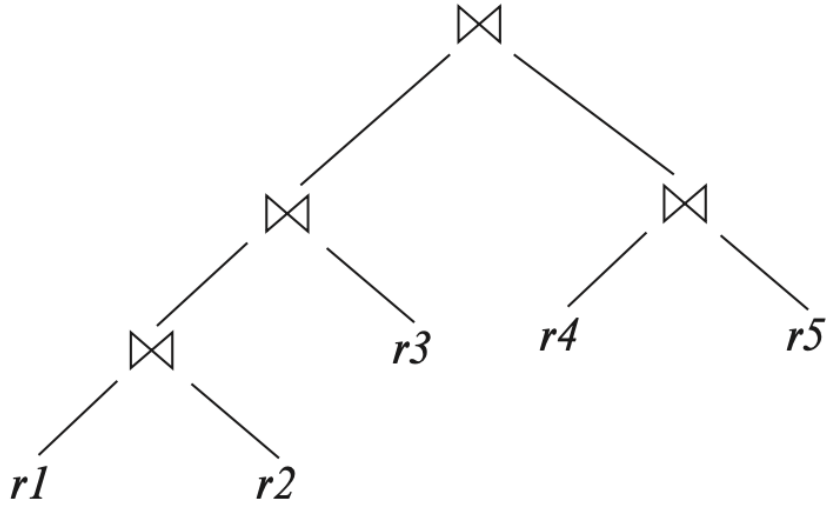
Xi'an Jiaotong-Liverpool University 西交利物浦大学

Other heuristics: Left Deep Join Trees

- In left-deep join tree, right-hand-side input for each join is relation, *not* result of intermediate join.



(a) Left-deep join tree



(b) Non-left-deep join tree

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Cost of Left Deep Join Optimisation

- To find best left-deep join tree for set of *n* relations:
  - Consider *n* alternatives with one relation as right-hand side input and other relations as left-hand side input.
  - *Modify dynamic programming optimisation algorithm*:
    - Replace "**for each** non-empty subset *S1* of *S* such that $S1 \neq S$"
    - By: ***for each** relation r in S, let S1 = S – r.*
- If only left-deep trees are considered, time complexity of finding best join order is $O(n!)$, with dynamic programming this can be reduced to $O(n\,2^n)$
  - Space complexity remains at $O(2^n)$
- Cost-based optimisation is expensive, but worthwhile for queries on large datasets (typical queries have small n, generally < 10)

Xi'an Jiaotong-Liverpool University 西交利物浦大学

Structure of Query Optimisers

- Many optimisers considers only left-deep join orders.
  - Plus heuristics to push selections and projections down query tree
  - Reduces optimisation complexity and generates plans amenable to pipelined evaluation.
- Even with use of heuristics, cost-based query optimisation imposes substantial overhead.
  - But worth for expensive queries
  - Optimisers often use simple heuristics for very cheap queries, and perform exhaustive enumeration for more expensive queries

Xi'an Jiaotong-Liverpool University 西交利物浦大学

End of Lecture

- Reading
  - Textbook chapter 16.1, 16.2, 16.3, 16.4